PROGRAMACIÓN III

Fundamentos de CSS y Diseño Web

1. Introducción y Contexto Histórico

CSS (siglas de *Cascading Style Sheets* u Hojas de Estilo en Cascada) es el lenguaje estándar utilizado para definir la presentación y apariencia visual de un documento escrito en HTML.

Mientras que HTML se encarga de la estructura semántica (el esqueleto), CSS es responsable de la "piel" y la experiencia visual. Nos permite pasar de páginas en blanco y negro a interfaces profesionales, permitiéndonos:

- **Estilizar:** Modificar colores, tipografías y tamaños para identidad visual.
- **Posicionar:** Organizar elementos en la pantalla de forma precisa (layouts).
- **Animar:** Crear transiciones y efectos visuales modernos.
- **Adaptar (Responsividad):** Ajustar el diseño para que funcione en móviles, tablets y escritorios .

1.1. Evolución del Estándar

Propuesto originalmente por **Håkon Wium Lie** en 1994, CSS ha evolucionado significativamente:

- **CSS1 (1996):** Primera recomendación oficial.
- **CSS2 (1998):** Introdujo posicionamiento y capas.
- **CSS3 (2011 en adelante):** La versión actual, que introdujo módulos independientes como animaciones, Flexbox y Grid, permitiendo diseños complejos sin depender de herramientas externas .

2. Sintaxis e Implementación

2.1. Anatomía de una Regla CSS

CSS funciona mediante **reglas**. Una regla se compone de un **selector** (que indica a qué elementos afectar) y un **bloque de declaración** (que define qué cambios aplicar) .

CSS

```
selector {
   propiedad: valor;  /* Ejemplo: color: red; */
   unidad: medida;    /* Ejemplo: width: 100px; */
}
```

Cada declaración debe terminar obligatoriamente con un punto y coma (;).

2.2. Tipos de Selectores Fundamentales

Para aplicar estilos con precisión, utilizamos diferentes tipos de selectores :

1. **Selector de Etiqueta (Tag):** Afecta a todas las instancias de una etiqueta HTML.
   - Ejemplo: p { ... } (afecta a todos los párrafos).
2. **Selector de Clase (Class):** Afecta a un grupo de elementos específicos que comparten el atributo class. Se define con un **punto** (.).
   - Ejemplo: .boton-rojo { ... }.
3. **Selector de ID:** Afecta a un único elemento con un atributo id específico. Se define con un **numeral/hash** (#).
   - Ejemplo: #header-principal { ... }.

2.3. Métodos de Inclusión

Existen tres formas de aplicar CSS, ordenadas de la menos a la más recomendada:

1. **Inline (En línea):** Se escribe dentro de la etiqueta HTML (<p style="...">). **No recomendado** porque mezcla contenido con presentación y dificulta el mantenimiento .
2. **Interno:** Se escribe dentro de una etiqueta <style> en el <head> del HTML. Útil para páginas únicas pero poco escalable.
3. **Externo (Estándar Profesional):** Se crea un archivo independiente (ej. style.css) y se vincula al HTML. Esto separa completamente la lógica visual de la estructural.

Vincular CSS externo:

HTML

```
<link rel="stylesheet" href="style.css">
```

La etiqueta <link> debe ir siempre dentro del <head> .

3. Tipografía y Estilizado de Texto

El texto es el componente principal de la web. CSS ofrece propiedades extensas para mejorar su legibilidad y estética .

3.1. Propiedades Esenciales

- **color:** Define el color del texto (puede usar nombres, Hexadecimal, RGB).
- **font-family:** Define la familia tipográfica. Se recomienda definir una "pila" de fuentes (fallback) por si la principal no carga (ej: Arial, sans-serif) .
- **font-size:** Controla el tamaño de la letra.
- **font-weight:** Define el grosor o "peso" de la letra. Puede ser normal, bold (negrita) o valores numéricos del 100 al 900 .
- **font-style:** Permite poner el texto en cursiva (italic) o normal.

3.2. Formato y Alineación

- **text-align:** Alinea el texto horizontalmente (left, right, center, justify).
- **text-decoration:** Añade líneas al texto. Comúnmente usado para subrayar (underline) o quitar el subrayado de enlaces (none).
- **text-transform:** Modifica la capitalización: uppercase (mayúsculas), lowercase (minúsculas) o capitalize.
- **line-height y letter-spacing:** Controlan la altura de la línea y el espaciado entre caracteres respectivamente, vitales para la legibilidad de párrafos largos.

4. Unidades de Medida en CSS

Las unidades determinan el tamaño de fuentes, márgenes, anchos y altos. Su correcta elección es clave para la **responsividad** y **accesibilidad** .

4.1. Unidades Absolutas

- **Píxel (px):** Representa un punto físico en la pantalla. Es una medida fija.
  - *Uso recomendado:* Bordes, sombras y detalles precisos que no deben cambiar de tamaño.
  - *Desventaja:* No escala bien si el usuario cambia la configuración de tamaño de fuente del navegador .

4.2. Unidades Relativas (Recomendadas)

Estas unidades se adaptan al contexto, permitiendo diseños fluidos.

- **em:** Es relativo al tamaño de fuente del **elemento padre**. Si el padre tiene 16px, 1em = 16px. Tiene un efecto de "cascada" que a veces complica los cálculos .
- **rem (Root EM):** Es relativo al tamaño de fuente del **elemento raíz** (html). Es la unidad estándar moderna para tipografía y espaciados generales porque es predecible y accesible .
- **Porcentaje (%):** Relativo al tamaño del contenedor padre. Fundamental para layouts fluidos.

4.3. Unidades de Viewport

Se basan en el tamaño visible de la ventana del navegador (Viewport).

- **vw (Viewport Width):** 1vw = 1% del ancho de la pantalla.
- **vh (Viewport Height):** 1vh = 1% del alto de la pantalla.
  - *Ejemplo:* height: 100vh hace que un elemento ocupe toda la altura visible de la pantalla .

5. El Modelo de Caja (Box Model)

En CSS, **absolutamente todos los elementos HTML son cajas rectangulares**. Entender cómo se comportan estas cajas es el fundamento del diseño web.

5.1. Componentes de la Caja

Una caja se compone de cuatro capas, desde adentro hacia afuera :

1. **Content (Contenido):** El elemento real (texto, imagen, video).
2. **Padding (Relleno):** Espacio **interno** transparente entre el contenido y el borde. "Infla" la caja.
3. **Border (Borde):** La línea que delimita la caja. Puede tener color, grosor y estilo (solid, dashed).
4. **Margin (Margen):** Espacio **externo** transparente que separa a la caja de otros elementos vecinos.

5.2. El problema del cálculo de tamaño (box-sizing)

Por defecto, CSS usa el modelo content-box. Esto significa que si defines width: 200px y agregas padding: 20px, el ancho total real en pantalla será de 240px (200 + 20 izq + 20 der). Esto suele romper los diseños .

La solución moderna: border-box Se utiliza la propiedad box-sizing: border-box. Con este modelo, el width que defines incluye automáticamente el contenido, el padding y el borde. Si defines 200px, la caja medirá 200px totales, y el navegador ajustará el contenido interno automáticamente. Esto permite un control preciso de las dimensiones .

5.3. Propiedades Visuales de la Caja

Además de dimensiones, podemos estilizar la caja con:

- **background-color:** Color de fondo.
- **border-radius:** Permite redondear las esquinas de la caja (ej. 0.4rem para botones suaves).
- **box-shadow:** Agrega sombras para dar profundidad y efecto de elevación.

PROGRAMACIÓN III

Teoría: CSS Intermedio y Layout

Esta sección profundiza en cómo controlar la visualización de elementos, posicionarlos en la pantalla, organizarlos mediante grillas (Grid) y mantener el código ordenado con metodologías profesionales (BEM).

1. Visualización y Visibilidad (Display & Visibility)

El comportamiento de las "cajas" en CSS depende fundamentalmente de la propiedad display.

Tipos de Display

- **Display Block (Bloque):** Es el comportamiento por defecto de elementos como , <h1>, o <p>. Estos elementos ocupan todo el ancho disponible de su contenedor y fuerzan un salto de línea, colocándose uno debajo del otro.
- **Display Inline (En línea):** Comportamiento típico de , <a> o . Se alinean en la misma línea horizontal junto al texto. Solo ocupan el ancho de su contenido y **ignoran** las propiedades de width y height si intentamos asignarlas.
- **Display Inline-Block:** Combina lo mejor de los dos anteriores. Permite que los elementos se alineen horizontalmente (como inline), pero respetando el ancho, alto, márgenes y relleno que definamos (como block).

Ocultar Elementos

Existen dos formas principales de ocultar elementos, con diferencias clave:

1. **Display None:** El elemento desaparece por completo del HTML y **no ocupa espacio**. Es útil para menús desplegables o elementos que no deben interferir en el diseño hasta que se activen.
2. **Visibility Hidden:** El elemento se oculta visualmente, pero **sigue ocupando su espacio** original en el flujo del documento, dejando un hueco vacío.

Opacidad

La propiedad opacity define la transparencia de un elemento en una escala del 0 al 1, donde 1 es visible y 0 es invisible (similar a desaparecer, pero manteniendo interactividad según el caso).

2. Posicionamiento y Capas (Position & Z-Index)

La propiedad position nos permite sacar elementos del flujo normal del documento y ubicarlos con precisión.

Tipos de Posicionamiento

- **Static:** Es el valor por defecto. Los elementos siguen el orden natural del HTML.
- **Relative:** El elemento se desplaza respecto a su posición original usando top, left, etc., pero **reserva su espacio original** en el documento. Además, sirve como referencia (contenedor padre) para los hijos con posición absoluta.
- **Absolute:** El elemento sale del flujo normal (no ocupa espacio). Se posiciona en relación a su ancestro posicionado más cercano (generalmente uno con position: relative). Si no hay ninguno, se posiciona respecto al body.
- **Fixed:** El elemento se fija a la ventana del navegador. No se mueve aunque hagamos scroll (útil para menús fijos o botones de chat).
- **Sticky:** Es un híbrido. Se comporta como relative hasta que hacemos scroll y llegamos a un punto específico, momento en el que se comporta como fixed (pegándose a la pantalla).

Z-Index (Control de Profundidad)

Controla el apilamiento de los elementos (cuál está encima de cuál).

- Funciona en elementos que **no** son static (tienen que tener relative, absolute, etc.).
- Un valor numérico mayor coloca el elemento por encima de uno con valor menor.

3. Selectores Avanzados

Para aplicar estilos con mayor precisión sin llenar el HTML de clases, usamos selectores avanzados.

- **Combinador Descendiente (div p):** Selecciona todos los párrafos (p) que estén dentro de un div, sin importar qué tan profundo estén anidados.
- **Combinador de Hijo Directo (div > p):** Selecciona únicamente a los hijos inmediatos. Si hay un párrafo dentro de otro contenedor dentro del div, este no será afectado.
- **Pseudoclases de Estado:**
  - :hover: Aplica estilos cuando el usuario pasa el cursor por encima (ej. cambiar color de botón).
  - :focus: Aplica estilos cuando un elemento recibe el foco (ej. al hacer clic en un input para escribir).
- **Pseudoclase Estructural (:nth-child):** Permite seleccionar elementos según su orden.
  - :nth-child(2): Selecciona el segundo hijo.
  - :nth-child(even) / :nth-child(odd): Selecciona elementos pares o impares.

4. CSS Grid Layout

CSS Grid es un sistema de diseño bidimensional que permite organizar elementos en filas y columnas simultáneamente, facilitando estructuras complejas y responsivas.

Conceptos Clave

- **Contenedor:** Se activa con display: grid en el elemento padre.
- **Columnas y Filas:** Se definen con grid-template-columns y grid-template-rows.
- **Unidad FR (Fracción):** Grid introduce la unidad fr, que representa una fracción del espacio disponible. Ejemplo: 1fr 1fr divide el espacio en dos partes iguales.
- **Función Repeat:** Para evitar repeticiones, usamos repeat(3, 1fr) para crear tres columnas iguales.
- **Gap (Hueco):** La propiedad gap (o grid-column-gap / grid-row-gap) define el espacio de separación entre las celdas.

Alineación

- justify-items: Alineación horizontal dentro de la celda (start, center, end, stretch).
- align-items: Alineación vertical dentro de la celda.
- place-items: Atajo para alinear en ambas direcciones a la vez.

Posicionamiento de Ítems

Podemos hacer que un elemento ocupe más de una celda usando líneas de referencia:

- grid-column-start / grid-column-end: Define dónde empieza y termina un elemento horizontalmente.
- grid-row-start / grid-row-end: Define dónde empieza y termina verticalmente.
- Esto permite que un elemento ocupe, por ejemplo, desde la línea 1 hasta la 3, abarcando dos columnas.

5. Metodología BEM (Block, Element, Modifier)

BEM nace para solucionar los problemas de escalabilidad en proyectos grandes, evitando selectores complejos y conflictos de estilos. Su objetivo es crear código modular y mantenible.

Los 3 Pilares

1. **Block (Bloque):** Es una entidad independiente con significado propio (ej. card, header, menu). No debe depender de otros elementos.
2. **Element (Elemento):** Una parte del bloque que no tiene sentido por sí sola. Se denota con **dos guiones bajos** __ (ej. card__title, menu__item).
3. **Modifier (Modificador):** Una variación en el estilo o estado de un bloque o elemento. Se denota con **dos guiones medios** -- (ej. card--active, btn--large).

Reglas de Oro

- Los modificadores **complementan**, no reemplazan. Se usan junto a la clase base (ej. <button class="btn btn--alert">).
- Evitar anidación excesiva en el nombre (no hacer bloque__elemento__subelemento). Si necesitas eso, probablemente necesites un nuevo bloque

PROGRAMACIÓN III

📘 Teoría: CSS Avanzado - Flexbox y Diseño Responsivo

En esta sección aprenderás a distribuir elementos de manera inteligente con **Flexbox** y a adaptar tus sitios web a cualquier dispositivo (móviles, tablets, escritorio) utilizando **Media Queries**.

1. Flexbox (Caja Flexible)

Flexbox es un modelo de diseño unidimensional que permite organizar elementos de forma eficiente y predecible, incluso cuando su tamaño es desconocido o dinámico. Su principal ventaja es que distribuye el espacio disponible de manera inteligente entre los elementos.

Activación y Ejes

Para usar Flexbox, se aplica la propiedad display: flex al contenedor padre.

- Por defecto, los elementos hijos se alinean uno al lado del otro (en fila).
- **Flex Direction:** Controla la dirección de los elementos.
    - row: De izquierda a derecha (valor por defecto).
    - row-reverse: De derecha a izquierda.
    - column: Un elemento debajo del otro.
    - column-reverse: Columna invertida (el primero pasa a ser el último).

Alineación y Distribución (Propiedades del Contenedor)

Flexbox facilita la alineación vertical y horizontal.

1. **Justify Content (Eje Principal/Horizontal):** Alinea los elementos a lo largo del eje principal.
    - flex-start: Al principio (izquierda).
    - flex-end: Al final (derecha).
    - center: Centra el contenido.
    - space-between: Separa los elementos al máximo dentro de la caja padre.
    - space-around: Deja espacio alrededor de los elementos, incluyendo los bordes.
    - space-evenly: Distribuye el mismo espacio entre todos los elementos.
2. **Align Items (Eje Cruzado/Vertical):** Alinea los elementos perpendicularmente al eje principal.
    - stretch: (Por defecto) Estira los ítems para ocupar todo el alto posible.
    - flex-start: Alinea arriba.
    - flex-end: Alinea abajo.

    - center: Centra verticalmente.
    - baseline: Alinea basándose en la línea base del texto.
3. **Otras Propiedades Clave:**
    - **Gap:** Define el espaciado o separación entre filas y columnas.
    - **Flex Wrap:** Permite que los elementos salten a la siguiente línea si no hay espacio suficiente en la pantalla.

Propiedades de los Ítems (Hijos)

Podemos controlar el comportamiento individual de cada elemento flexible.

- **Flex Grow:** Define la capacidad de un ítem para crecer si hay espacio disponible (ej. ocupar el espacio sobrante).
- **Flex Shrink:** Determina cuánto puede encogerse un ítem si el espacio es insuficiente. Un valor más alto significa mayor capacidad de encogimiento.
- **Flex Basis:** Define el tamaño inicial del ítem antes de aplicar crecimiento o encogimiento.
- **Align Self:** Permite alinear un ítem individual de forma diferente al resto (ej. todos arriba, pero uno abajo).

2. Media Queries y Diseño Responsivo

Las Media Queries son reglas de CSS que aplican estilos condicionales según las características del dispositivo o preferencias del usuario. Esto permite adaptar el diseño a diferentes tamaños de pantalla, orientaciones o resoluciones.

Sintaxis Básica

Se utiliza la regla @media seguida de una condición. Los estilos dentro de esa regla solo se aplican si la condición se cumple.

CSS
```
@media (max-width: 768px) {
  /* Estilos para pantallas de 768px o menos */
  body { background-color: lightblue; }
}
```

Breakpoints (Puntos de Ruptura)

Son los anchos de pantalla estándar donde el diseño cambia para adaptarse.

- **Móvil:** Generalmente hasta 480px.
- **Tablet:** Entre 768px y 1024px.
- **Escritorio (Desktop):** Desde 1025px en adelante.

Estrategia Mobile First

Esta filosofía sugiere diseñar y maquetar primero para dispositivos móviles (pantallas pequeñas) y luego ir adaptando y agrandando el contenido para pantallas más grandes. Esto resulta en un código más limpio y eficiente.

Tipos de Consultas

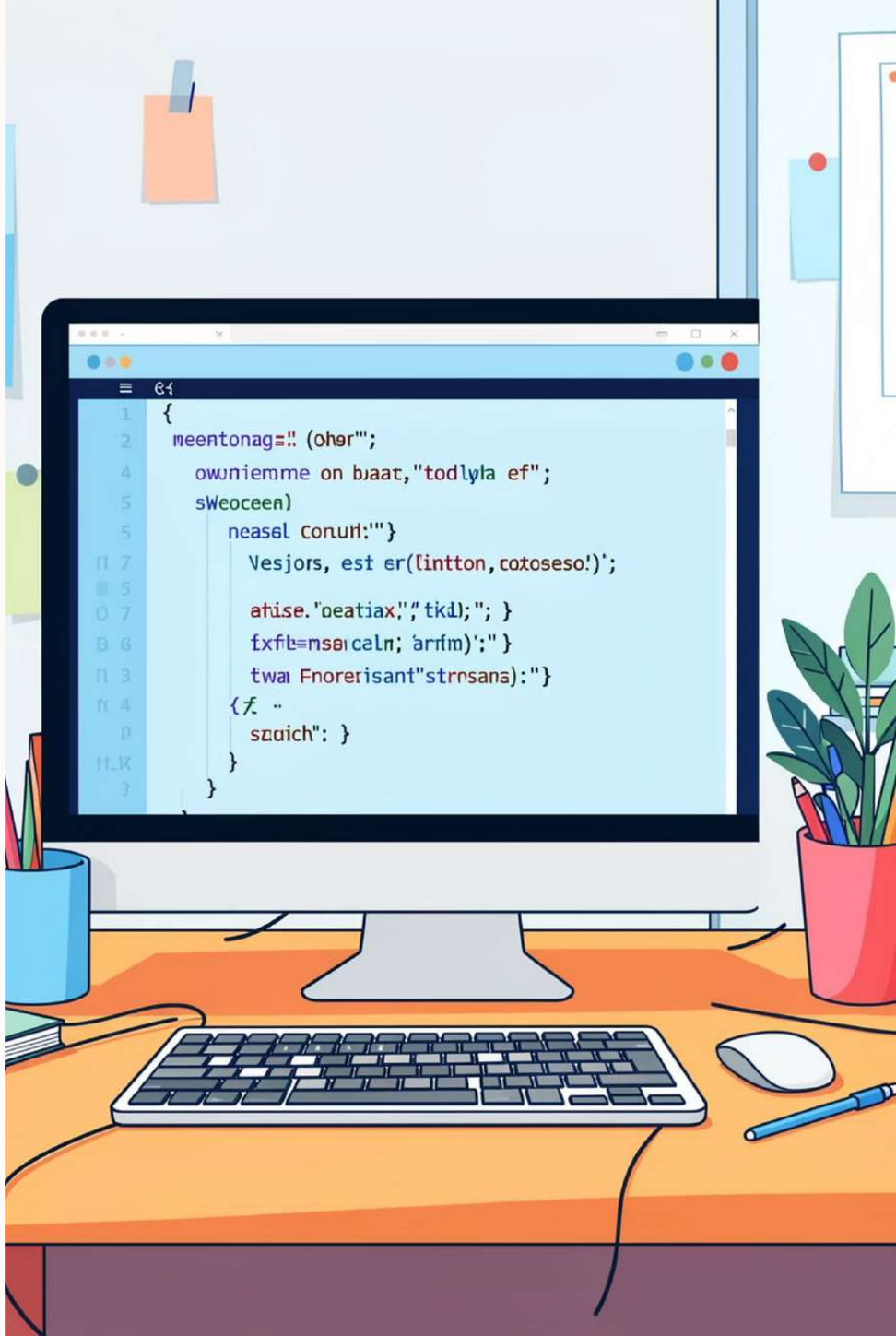
- **Por Tamaño:** Usando min-width (ancho mínimo) o max-width (ancho máximo).
- **Por Orientación:** Detectar si el dispositivo está en vertical (portrait) u horizontal (landscape) para cambiar la disposición (ej. pasar de 1 columna a 3).
- **Preferencias de Usuario (Accesibilidad):**
    - prefers-color-scheme: Detectar tema oscuro o claro.
    - prefers-reduced-motion: Desactivar animaciones y transiciones para usuarios sensibles al movimiento.
    - prefers-contrast: Ajustar el contraste.

Buenas Prácticas

1. **Usar unidades relativas:** Emplear %, rem, em o vh/vw en lugar de píxeles fijos para mayor adaptabilidad.
2. **Probar en dispositivos reales:** No confiar solo en el redimensionamiento del navegador.
3. **Organización:** Definir breakpoints claros y lógicos.

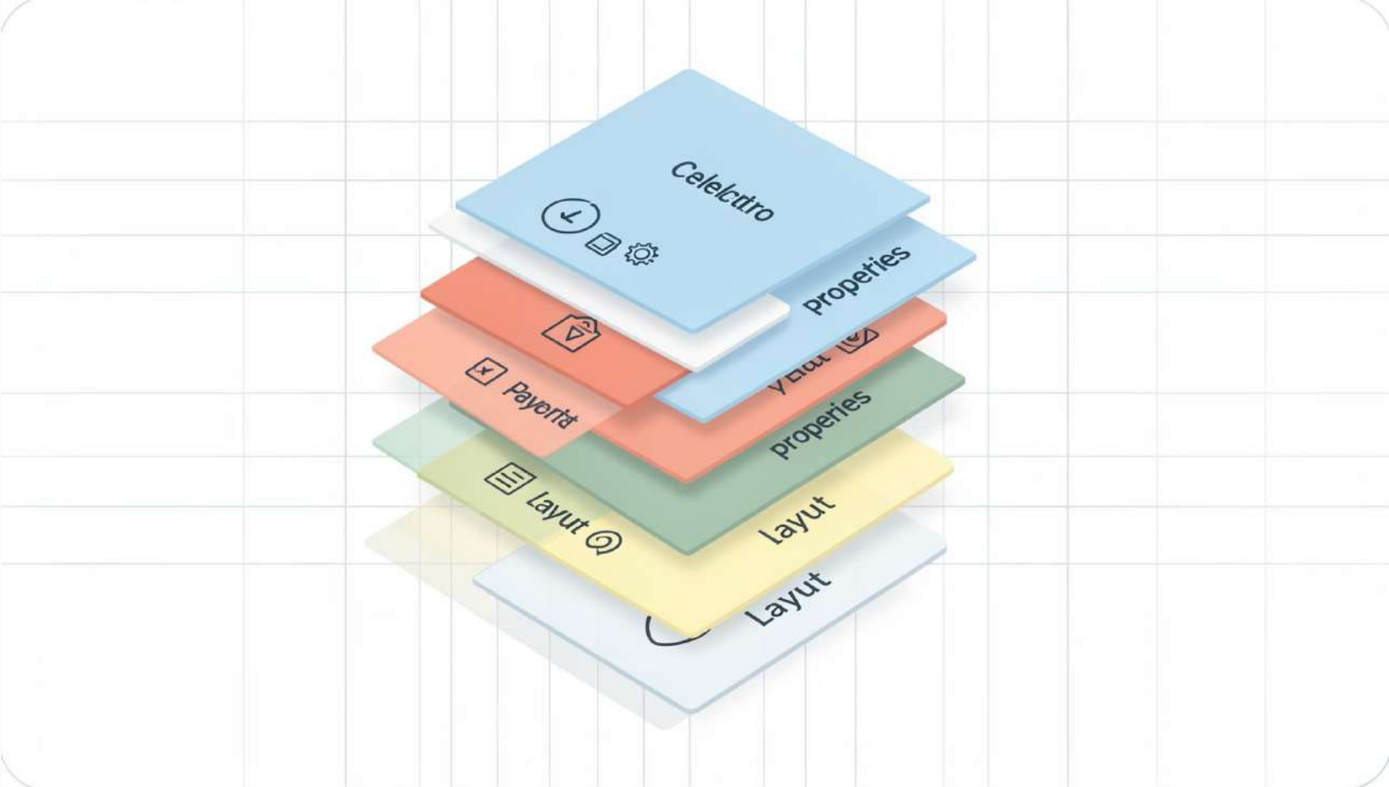
Fundamentos de CSS y Diseño Web

Programación III - Hojas de Estilo en Cascada



INTRODUCCIÓN

¿Qué es CSS?



CSS (Cascading Style Sheets) es el lenguaje estándar para definir la presentación visual de documentos HTML. Mientras HTML estructura el contenido, CSS define su apariencia.

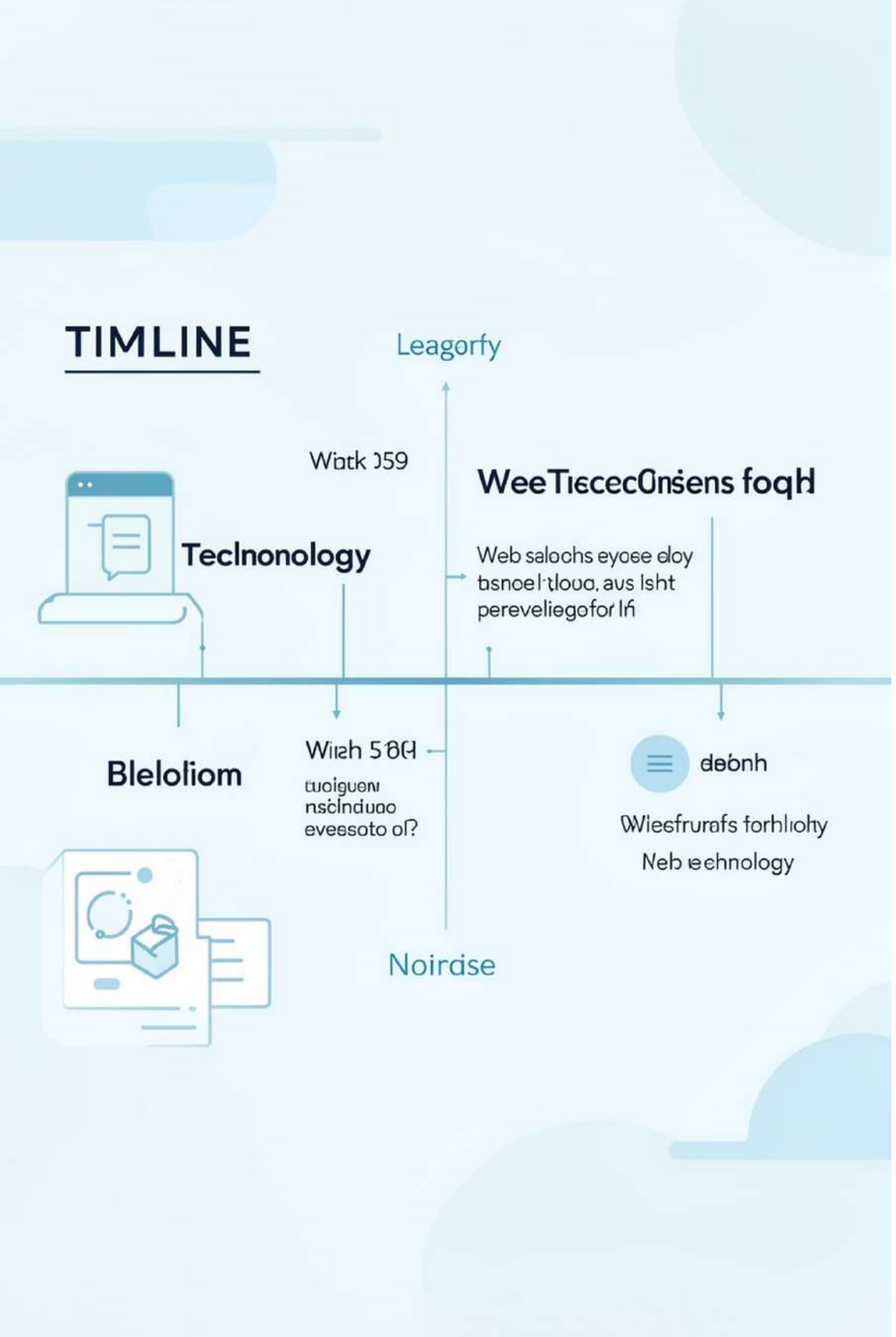
Estilizar
Colores, tipografías y tamaños

Posicionar
Organizar elementos con precisión

Animar
Transiciones y efectos visuales

Adaptar
Diseño responsivo multiplataforma

Made with GAMMA



Evolución del Estándar CSS

1 **CSS1 (1996)**

Primera recomendación oficial propuesta por Håkon Wium Lie

2 **CSS2 (1998)**

Introdujo posicionamiento y sistema de capas

3 **CSS3 (2000+)**

Versión modular actual con Flexbox, Grid y animaciones

Made with GAMMA

SINTAXIS

Anatomía de una Regla CSS

Estructura Básica

Una regla CSS se compone de un **selector** que indica qué elementos afectar y un **bloque de declaración** que define los cambios a aplicar.

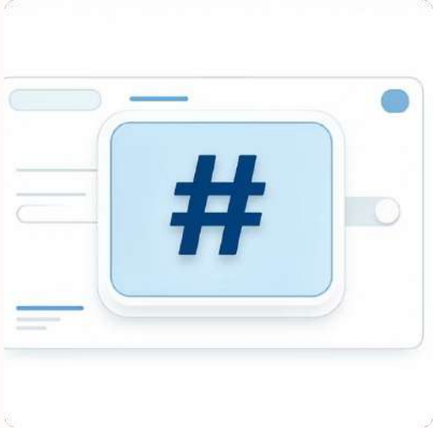
Cada declaración debe terminar obligatoriamente con punto y coma (;)

```
selector {
 propiedad: valor;
 unidad: medida;
}

/* Ejemplo práctico */
p {
 color: red;
 width: 100px;
}
```

Made with GAMMA

Tipos de Selectores Fundamentales



Selector de Etiqueta

Afecta a todas las instancias de una etiqueta HTML. Ejemplo: `p { ... }`

Selector de Clase

Afecta a elementos con atributo `class`. Se define con punto: `.boton-rojo`

Selector de ID

Afecta a un único elemento con `id` específico. Se define con #: `#header-principal`

Made with GAMMA

Métodos de Inclusión de CSS

Inline (En línea)

Dentro de la etiqueta HTML. No recomendado - mezcla contenido con presentación.

Interno

Etiqueta `<style>` en el `<head>`. Útil para páginas únicas.

Externo (Estándar)

Archivo independiente vinculado. Método profesional que separa lógica visual.

Made with GAMMA

Tipografía y Estilizado de Texto

El texto es el componente principal de la web. CSS ofrece propiedades extensas para mejorar legibilidad y estética.

color

Define el color del texto (nombres, Hex, RGB)

font-family

Familia tipográfica con fallback

font-size

Tamaño de la letra

font-weight

Grosor (100-900, normal, bold)

text-align

Alineación horizontal del texto

text-decoration

Líneas decorativas (underline, none)

Made with GAMMA

MEDIDAS

Unidades de Medida en CSS

Unidades Relativas

Recomendadas para diseños responsivos y accesibles

- **rem:** Relativo al elemento raíz. Estándar moderno predecible
- **em:** Relativo al padre. Efecto cascada
- **%:** Relativo al contenedor. Layouts fluidos
- **vw /vh:** Basado en viewport (1vw = 1% ancho pantalla)

Unidades Absolutas

px (píxel): Punto físico fijo en pantalla

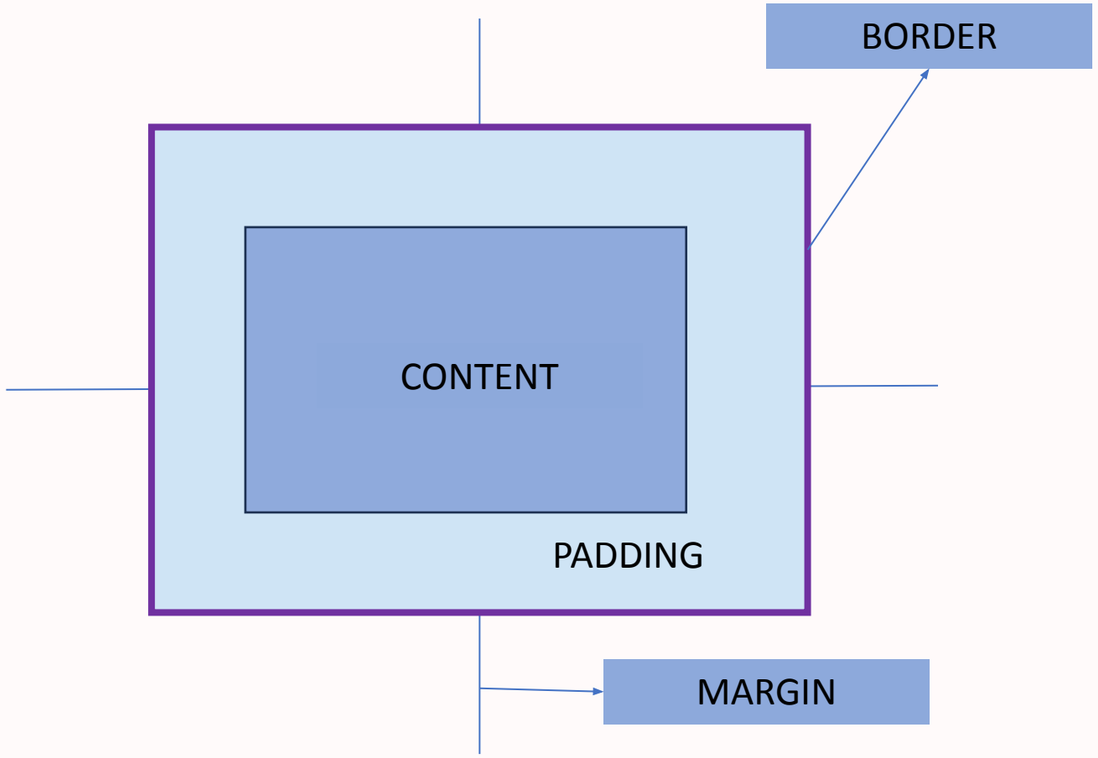
Uso recomendado: Bordes, sombras y detalles precisos

Desventaja: No escala si el usuario cambia configuración de tamaño de fuente

Made with GAMMA

El Modelo de Caja (Box Model)

En CSS, todos los elementos HTML son cajas rectangulares. Entender su comportamiento es fundamental para el diseño web.



0 1

Content (Contenido)

El dato en sí: texto, imagen, video

0 2

Padding (Relleno)

Espacio interno transparente que "infla" la caja

0 3

Border (Borde)

Línea delimitadora con color, grosor y estilo

0 4

Margin (Margen)

Espacio externo que separa elementos vecinos

Made with GAMMA

SOLUCIÓN MODERNA

Box-Sizing: Border-Box

El Problema

Por defecto, CSS usa `content-box`. Si defines `width: 200px` y agregas `padding: 20px`, el ancho total será 240px, rompiendo diseños.

La Solución

`box-sizing: border-box` incluye automáticamente contenido, padding y borde en el width definido. Control preciso de dimensiones.

Propiedades Visuales

background-color

Color de fondo de la caja

border-radius

Redondea esquinas (ej. 0.4rem)

box-shadow

Sombras para profundidad y elevación

Made with GAMMA

Visualización y Visibilidad (Display & Visibility)

Aprende a controlar cómo se muestran y ocupan espacio tus elementos en el diseño web, fundamental para layouts y efectos interactivos.

Propiedad `display`: Control del Flujo

Block (Bloque)

Ocupa todo el ancho disponible y fuerza un salto de línea.

`DIV, P, H1`

Inline (En línea)

Ocupa solo el ancho de su contenido, sin salto de línea.

`SPAN, A, STRONG`

Inline-Block

Flujo en línea pero acepta ancho, alto y márgenes verticales.

`IMG, BUTTON`

Ocultando Elementos: `display: none` vs `visibility: hidden`

`display: none`

Elimina el elemento del flujo del documento, como si nunca existió.

- No ocupa espacio.
- No es interactivo.
- Útil para componentes que aparecen y desaparecen completamente.

`visibility: hidden`

Oculta el elemento, pero mantiene su espacio en el layout.

- Mantiene el espacio que ocuparía.
- No es interactivo.
- Útil para animaciones de entrada/salida donde el espacio es fijo.

Opacidad: Transparencia Controlada

La propiedad `opacity` controla el nivel de transparencia de un elemento. Acepta valores de `0` (completamente transparente) a `1` (completamente opaco).

Made with GAMMA

Posicionamiento y Capas (Position & Z-Index)

Comprender cómo posicionar elementos en CSS es crucial para crear diseños complejos y adaptables. Controlar la ubicación y el orden de apilamiento es fundamental para la disposición visual.

Static
El valor por defecto. Los elementos se posicionan según el flujo normal del documento.

Relative
Se posiciona con relación a su propia posición normal, dejando un hueco en su lugar original.

Absolute
Se posiciona con respecto al ancestro posicionado más cercano. Sale del flujo normal.

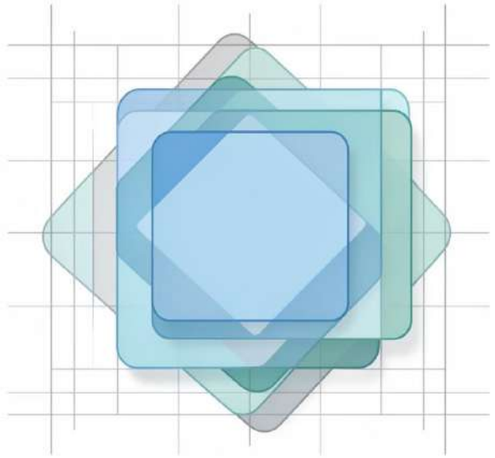
Fixed
Se posiciona con respecto a la ventana gráfica del navegador. Permanece en su lugar al desplazarse.

Sticky
Híbrido entre `relative` y `fixed`. Permanece en el flujo hasta alcanzar un umbral.

Control de Profundidad (Z-Index)

La propiedad `z-index` controla la superposición de elementos posicionados. Un valor más alto significa que el elemento se mostrará **por encima** de otros.

Solo funciona en elementos con una propiedad `position` distinta de `static`.



Made with GAMMA

AVANZADO

Selectores Avanzados

Más allá de las etiquetas y clases, los selectores avanzados permiten un control granular y potente sobre los elementos HTML, facilitando estilos complejos y dinámicos.

Combinadores
Relaciones entre elementos

Pseudoclases de Estado
Interacciones y condición

Pseudoclases Estructurales
Posición y orden

Combinadores

Permiten seleccionar elementos basándose en su relación con otros elementos.

- `selector1 selector2`: Descendiente (cualquier nivel)
- `selector1 > selector2`: Hijo directo (un nivel)

```
/* Todos los párrafos dentro de un div */
div p {
  color: blue;
}

/* Solo los 'li' que son hijos directos de 'ul' */
ul > li {
  list-style-type: square;
}
```

Pseudoclases de Estado

Seleccionan elementos cuando se encuentran en un estado particular, como :hover o :focus.

- `:hover`: Cuando el cursor está sobre el elemento.
- `:focus`: Cuando un elemento (como un input) está

```
/* Cambiar color de un botón al pasar el mouse */
button:hover {
  background-color: darkblue;
  color: white;
}

/* Estilo de un campo de texto al estar activo */
input:focus {
  border: 2px solid lightblue;
  outline: none; /* Elimina el contorno predeterminado del
navegador */
}
```

Pseudoclases Estructurales

Permiten seleccionar elementos basándose en su posición dentro de su elemento padre.

- `:nth-child(n)`: Selecciona el enésimo hijo.
- `:first-child`: Selecciona el primer hijo.
- `:last-child`: Selecciona el último hijo.

```
/* Estilo para cada segundo elemento de una lista */
li:nth-child(even) {
  background-color: #f0f0f0;
}

/* El primer elemento de un menú */
nav li:first-child {
  font-weight: bold;
}
```

Made with GAMMA

CSS Grid Layout

CSS Grid es un sistema de maquetación bidimensional diseñado para la creación de interfaces complejas y responsive de manera eficiente, controlando tanto las columnas como las filas de un diseño.

Conceptos Fundamentales

- **Grid Container:** El elemento padre que se convierte en una cuadrícula (`display: grid;`).
- **Grid Items:** Los hijos directos del contenedor, que se posicionan dentro de la cuadrícula.
- **Columnas y Filas:** Definidas por el usuario, ya sean explícitas o implícitas.

Utiliza la unidad `fr` para fracciones de espacio disponible y la función `repeat()` para crear patrones de columnas y filas de forma concisa. La propiedad `gap` define el espacio entre las celdas de la cuadrícula.

Alineación de Ítems

Controla cómo se posicionan los elementos dentro de sus celdas:

- `justify-items` (horizontal)
- `align-items` (vertical)
- `place-items` (ambos a la vez)

Posicionamiento de Ítems

Define dónde se ubican los ítems en la cuadrícula:

- `grid-column-start/end`: Inicio y fin en columnas.
- `grid-row-start/end`: Inicio y fin en filas.
- Shorthands: `grid-column` y `grid-row`.

Made with GAMMA

ESTRUCTURA

Metodología BEM (Block, Element, Modifier)

BEM es una convención de nomenclatura que ayuda a los desarrolladores a construir interfaces de usuario modulares, escalables y reutilizables. Organiza tu CSS para facilitar el mantenimiento y la colaboración en grandes proyectos web.

Block (Bloque)

Una entidad independiente y autónoma de la interfaz. Puede existir por sí sola y reutilizarse en cualquier parte del proyecto. Representa un componente completo.

```
.card
.button
.header
```

Element (Elemento)

Una parte de un bloque que no tiene significado independiente. Siempre forma parte de un bloque y se une con `__` (dos guiones bajos).

```
.card__title
.button__icon
.header__logo
```

Modifier (Modificador)

Una bandera que altera la apariencia o el comportamiento de un bloque o elemento. Se une con `--` (dos guiones medios).

```
.button--primary
.card--large
.button__icon--small
```

Reglas de Oro y Buenas Prácticas

- El CSS es **plano**: Evita la anidación profunda en los selectores.
- Los nombres de clase deben ser **significativos** y descriptivos.
- Un bloque **no debe depender de otros bloques** en su CSS.
- Los modificadores se usan para **variaciones**, no para añadir funcionalidad nueva.
- Mantén la **consistencia** en la nomenclatura de todo el equipo.

Ejemplos de Nomenclatura

✅ Correcto

```
.menu
.menu__item
.menu__item--active
.button
.button--disabled
```

❌ Incorrecto

```
.main-nav-item
.menu .item /* Anidación */
.btn-red /* Color específico */
#header-logo /* Usar ID */
.button.is-disabled /* Modificador doble */
```

Made with GAMMA

Flexbox (Caja Flexible)

Flexbox es un modelo de diseño unidimensional que permite distribuir y alinear elementos en un contenedor de manera eficiente, ideal para construir interfaces responsivas.

Activación

Aplica `display: flex;` al contenedor para convertirlo en un **Flex Container**. Sus hijos directos se convierten en **Flex Items**.

Dirección

`flex-direction` establece el eje principal. Valores:

- `row` (por defecto): Horizontal
- `column`: Vertical
- `-reverse`: Invierte el orden

Alineación

Controla la distribución de ítems:

- `justify-content`: Alinea en el eje principal.
- `align-items`: Alinea en el eje transversal.

Flujo y Espacio

- `flex-wrap`: Permite que los ítems salten de línea.
- `gap`: Define el espacio entre ítems (filas y columnas).

Made with GAMMA

FLEXBOX AVANZADO

Propiedades de los Ítems en Flexbox

Para un control granular del comportamiento de cada elemento flexible dentro de un contenedor, se utilizan estas propiedades clave:

`flex-grow`

Define cuánto crecerá un ítem en proporción al espacio libre disponible. Un valor de `1` le permite crecer equitativamente con otros ítems.

`flex-shrink`

Determina la capacidad de un ítem para encogerse. Por defecto, los ítems se encogen (`1`). Un valor de `0` impide que el ítem se encoja.

`flex-basis`

Especifica el tamaño inicial del ítem antes de que el espacio restante se distribuya. Puede ser un valor como `200px` o `auto`.

`align-self`

Alinea un ítem individual a lo largo del eje transversal, anulando la propiedad `align-items` del contenedor flex.

Made with GAMMA

Media Queries y Diseño Responsivo

Adaptamos nuestros sitios web para una experiencia óptima en cualquier dispositivo, garantizando que tu contenido luzca impecable en móviles, tablets y escritorios.

Sintaxis Básica y Breakpoints

Las Media Queries son reglas CSS condicionales que aplican estilos específicos cuando se cumplen ciertas condiciones del dispositivo o entorno de visualización.

La sintaxis fundamental es `@media (condición) { /* estilos */ }`.

Los **breakpoints** son los puntos de ruptura en los cuales el diseño de una página web se ajusta. Algunos estándares comunes son:

- **Móviles:** Hasta 767px
- **Tablets:** Entre 768px y 1023px
- **Escritorio:** Desde 1024px en adelante

MOBILE-FIRST

La estrategia **Mobile-First** prioriza la experiencia móvil, aplicando estilos base para pantallas pequeñas y luego añadiendo adaptaciones para dispositivos más grandes.

```
/* Estilos por defecto para móviles */
body {
  font-family: sans-serif;
  font-size: 16px;
  margin: 10px;
}
/* Tablets en adelante */
@media (min-width: 768px) {
  body {
    font-size: 18px;
    margin: 20px;
  }
  .nav {
    display: flex;
    justify-content: space-around;
  }
}
/* Desktops en adelante */
@media (min-width: 1024px) {
  body {
    font-size: 20px;
  }
  .container {
    max-width: 1200px;
    margin: 0 auto;
```

Made with GAMMA

Buenas Prácticas en Diseño Responsivo

El diseño responsivo es clave para una experiencia de usuario consistente en cualquier dispositivo. Aplicar estas prácticas asegura que tu sitio sea adaptable y accesible.

Priorizar Unidades Relativas

Usa `%`, `rem`, `em`, `vw/vh` para asegurar la escalabilidad de elementos, tipografías y espaciados en diferentes tamaños de pantalla.

Probar en Dispositivos Reales

Las simulaciones de navegador son útiles, pero la validación final debe hacerse en una variedad de **dispositivos físicos** y contextos de uso.

Estrategia de Breakpoints

Define puntos de ruptura (**breakpoints**) basados en el contenido y cómo se comporta, no solo en tamaños de pantalla predefinidos, para una adaptación natural.

Enfoque Mobile-First

Prioriza el diseño **móvil primero para pantallas pequeñas**, luego escala progresivamente a tamaños mayores. Esto mejora el rendimiento y la experiencia móvil.

Optimización de Medios

Implementa imágenes responsivas con `srcset` y `<picture>`, y optimiza videos para cargar eficientemente, garantizando la velocidad de carga.

Made with GAMMA

PROGRAMACIÓN III

Trabajo Práctico Integrador: Página de Login - Food Store

OBJETIVO GENERAL

Aplicar los conceptos aprendidos de CSS (selectores, modelo de caja, propiedades de texto, display y variables) para maquetar correctamente una página de login profesional para el proyecto **Food Store**.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
Modelo de Caja (Box Model)	El formulario de login se construye con margin, padding, border, border-radius y box-shadow.
Display y centrado	Uso de técnicas para centrar el formulario (tarjeta) en el medio de la página.
Propiedades de Texto	Uso de text-transform, line-height, letter-spacing, color y text-align para mejorar la legibilidad.
Variables CSS	Mantener consistencia en colores y permitir cambiar temas fácilmente.
Pseudoclases	Uso de :hover y :focus para generar interactividad en inputs y botones.
Estilizado de formularios	Aplicar estilos profesionales a input, label y button.

Caso Práctico

Se deberá crear un archivo llamado login.html con su respectivo login.css. El documento debe contener un formulario de login centrado que incluya:

1. **Logo o título** de "Food Store".
2. **Subtítulo** (ej: "Iniciar Sesión").
3. **Campo de email** (input type="email").
4. **Campo de contraseña** (input type="password").
5. **Botón de "Ingresar"** que cambie de color al pasar el mouse.
6. **Enlace** a "¿No tenés cuenta? Registrate".

Variables css para trabajar:

```
:root {
   --color-primario: #ff6347;
   --color-fondo: #f0f0f0;
   --color-texto: #333;
   --color-blanco: #fff;
   --color-borde: #ddd;
   --color-focus: #ff4500;
}
```

Requisitos Técnicos

HTML:

- Usar la etiqueta <form> con inputs correctamente estructurados.
- Cada input debe tener su <label> asociado (usando el atributo for).
- Usar tipos de input apropiados (email, password).
- Incluir atributos required y placeholder.

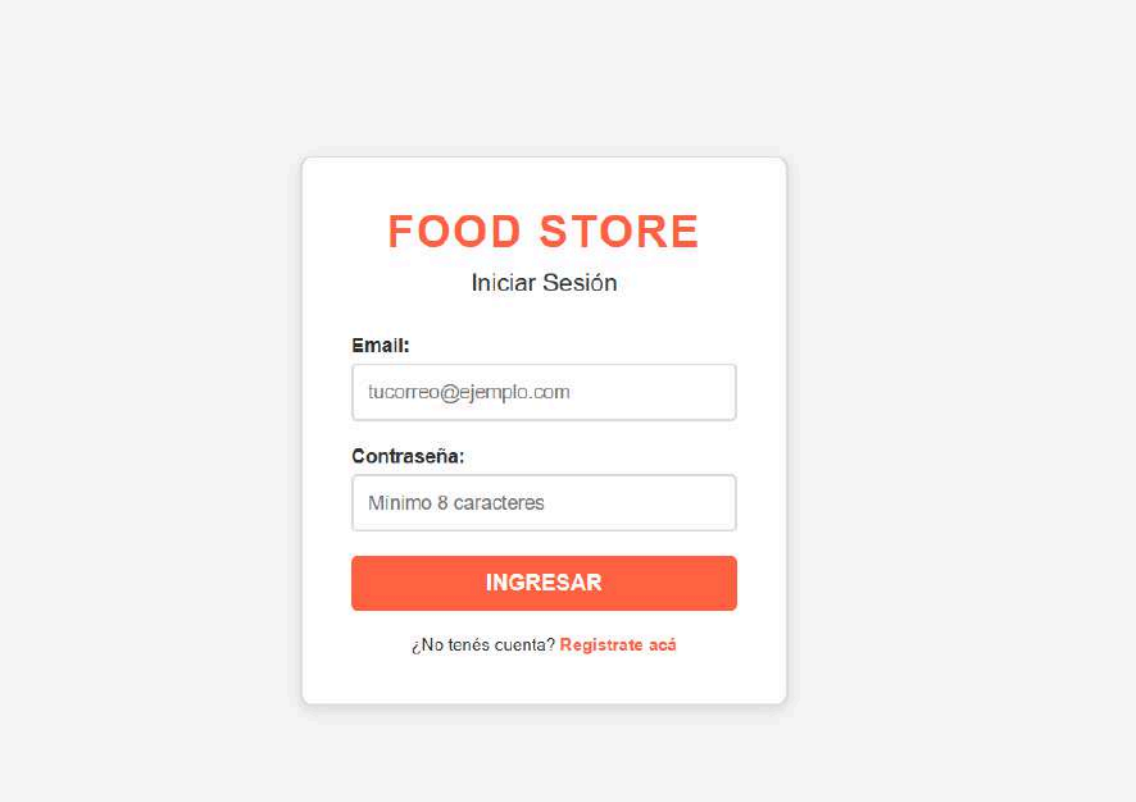
CSS:

- ✅ Usar **variables CSS** para los colores (fondo, texto, botones, bordes).
- ✅ Aplicar el **modelo de caja** (border, padding, margin, border-radius, box-shadow) al contenedor del formulario.
- ✅ **Centrar** el formulario en la página (puede ser usando Flexbox o márgenes automáticos).
- ✅ **Estilizar los inputs** (border, padding, width: 100%).
- ✅ Aplicar **:focus** en los inputs (cambiar el color del borde al hacer clic).

- ✅ Aplicar **:hover** en el botón (cambio visual al pasar el mouse).
- ✅ Usar text-transform: uppercase para el título principal.

Estructura Visual Esperada

CONCLUSIONES ESPERADAS

- Comprender cómo maquetar formularios completos en HTML y CSS.
- Aplicar correctamente el modelo de caja con margin, padding, border y box-shadow.
- Centrar elementos en la página usando técnicas CSS.
- Reconocer el valor de las variables CSS para unificar estilos.
- Aplicar pseudoclases (:hover, :focus) para mejorar la interactividad.
- Estilizar inputs y botones de forma profesional.

Ejemplo de vista